

第 41 讲

Overfitting 与 Generalization

Training Loss 很低，为什么模型仍然可能没有真正学会？

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

前面第 33–40 讲主要回答的是：模型能不能被优化起来？从本讲开始，问题变成：模型在训练集上学得很好，能不能把这种能力带到没有见过的数据上？这是从 Optimization 走向 Generalization 的分水岭。对科研来说，一个模型是否“有价值”，通常不取决于 Training Loss 能压多低，而取决于独立数据、独立任务、独立环境上是否仍然成立。

学习目标

学完本讲，你应该能够：

1. 严格区分 Optimization Success、Training Fit 与 Generalization；
2. 解释 Underfitting、Good Fit、Overfitting 三种状态分别长什么样；
3. 理解 Training Loss、Validation Loss、Test Loss 的角色差异；
4. 解释 Generalization Gap 是什么，以及它能说明什么、不能说明什么；
5. 从模型容量、数据量、噪声、训练时长等角度理解为什么会过拟合；
6. 看懂典型 Train/Validation 曲线，并识别“训练继续变好、验证开始变差”的过拟合信号；
7. 理解 Early Stopping 为什么本质上是一种模型选择策略；
8. 理解为什么高容量网络甚至可以记住 Random Labels；
9. 在 LLM / Transformer / Agent Systems 论文中识别 memorization、benchmark contamination、trajectory leakage 等泛化风险；
10. 为后面的 Train/Validation/Test、Regularization、Dropout、Normalization 建立正确的总框架。

1 先把两个问题彻底分开：能优化，和能泛化

前面我们反复做过：

$Forward \rightarrow Loss \rightarrow Backward \rightarrow Optimizer \rightarrow Parameter Update$

如果 Training Loss 一直下降，说明当前训练过程至少完成了一件事：

当前模型越来越适合当前训练数据

但是科研真正关心的通常是另一个问题：

面对没有参与训练的新数据，模型还能不能表现好？

这就是 **Generalization**（泛化）。

一句话区分

Optimization 问的是：“怎样把训练目标降下来？”

Generalization 问的是：“训练得到的规律能不能迁移到同分布但未见过的新样本？”

假设一个模型把 100 道训练题全部背下来了：

$$Training\ Accuracy = 100\%$$

如果换成同类型的 100 道新题以后只答对一半，那么“训练成功”并不能推出“真正学会”。

2 从数学上看：Training Loss 只是有限样本上的经验风险

设训练集共有 N 个样本：

$$D_{train} = \{(x_i, y_i)\}_{i=1}^N$$

模型参数为 θ ，单样本 Loss 为：

$$\ell(f_{\theta}(x_i), y_i)$$

训练时实际最小化的通常是：

$$R_{train}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i)$$

这叫做有限训练集上的 **Empirical Risk**（经验风险）。

真正希望模型在未来样本上表现好，对应的是某个未知数据分布 $P(x, y)$ 上的平均风险：

$$R_{pop}(\theta) = \mathbb{E}_{(x,y) \sim P} [\ell(f_{\theta}(x), y)]$$

我们通常不知道真实 P ，也无法精确算这个期望，所以才需要独立的 Validation/Test 数据去近似观察。

底层矛盾

训练只看到了有限样本，但我们希望它对“未来没有见过的样本”也成立。**Overfitting** 的本质，就是模型把有限训练样本中特有的细节也当成了值得学习的规律。

3 Underfitting、Good Fit、Overfitting 到底分别是什么？

为了避免只背词，我们从训练表现与新数据表现同时判断。

1. Underfitting (欠拟合)

模型连训练数据里的主要规律都没有学好：

$$\text{Train Loss high,} \quad \text{Validation Loss high}$$

常见原因：

- 模型容量过低；
- 特征不足；
- 训练时间太短；
- 优化失败；
- Learning Rate / Initialization 等配置不合理。

2. Good Fit (较合理拟合)

训练集学得较好，新数据也保持相近表现：

$$\text{Train Loss low,} \quad \text{Validation Loss low}$$

此时仍不能说“绝对正确”，但至少说明当前验证分布上的泛化较好。

3. Overfitting (过拟合)

训练集越来越好，但未见数据明显更差：

$$\text{Train Loss very low,} \quad \text{Validation Loss higher}$$

典型现象不是“训练失败”，恰恰相反：

Training Loss 可能继续漂亮地下降

问题发生在训练集之外。

本讲实验：同一数据、不同模型容量

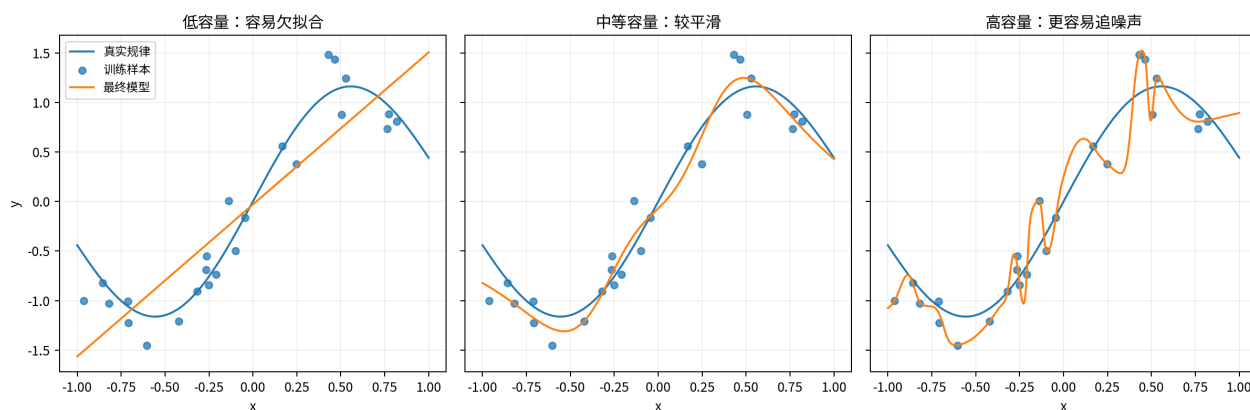
我们用 24 个带噪声训练点学习：

$$y = \sin(3x) + 0.3x + \text{noise}$$

训练 2000 Epoch 后：

模型	Final Train MSE	Final Validation MSE	Final Test MSE
Linear	0.163446	0.244899	0.238305
Small MLP	0.032329	0.104096	0.111837
Large MLP	0.002010	0.152615	0.161832

Large MLP 的 Training Loss 最低，但最终未见数据并不是最好。



4 为什么模型容量越大，越容易出现过拟合？

Model Capacity（模型容量） 可以通俗理解为：模型能够表达多复杂函数的能力。

一个简单线性模型：

$$y = wx + b$$

只能表达直线，因此面对复杂非线性规律可能欠拟合。

一个拥有大量隐藏单元和参数的 MLP 可以表达复杂得多的函数。优点是能拟合真正复杂的规律；风险是它也有能力去拟合：

- 测量噪声；
- 偶然异常点；
- 训练集中特有的巧合；
- 样本 ID、格式、模板等 shortcut；
- 数据泄漏带来的伪规律。

高容量不是“坏”

不能得出“模型越小越好”。容量太小会欠拟合。真正问题是：**模型容量、数据量、噪声、Regularization、训练预算之间是否匹配。**

5 数据量、模型容量和噪声：三个量要放在一起看

只说“模型很大，所以会过拟合”是不够准确的。更合理的理解是看三者的相对关系：

模型容量 vs. 有效数据量 vs. 噪声与任务复杂度

假设一个模型有能力表达非常复杂的函数，但训练集只有 20 个点，其中还包含测量噪声。此时很多完全不同的函数都可能穿过这 20 个点。训练数据本身不足以告诉模型“哪条复杂曲线才是真正常

律”。

如果数据量增加到 20 万个、覆盖范围更广，而且每种情况重复出现，那么许多只适合个别样本的偶然模式就更难持续成立。也就是说，**更多有效数据本身就可以起到约束模型的作用**。

一个直觉例子

只有两个点：

$$(0, 0), \quad (1, 1)$$

一条直线 $y = x$ 可以穿过它们；无数条复杂曲线也可以穿过它们。仅凭两个点，我们无法确定未来 $x = 0.5$ 时真正应该是多少。

如果再给出大量中间点都接近 $y = x$ ，那么“乱弯的复杂曲线”就越来越难同时解释所有数据。数据在事实上缩小了可接受解的范围。

所以模型容量本身不是罪魁祸首。现代 LLM 的 Parameter 数量极大，但同时也使用巨大数据、Regularization、预训练目标以及复杂训练 recipe。判断 Overfitting 时必须看整个系统，而不是只数 Parameter。

6 Overfitting 与 Distribution Shift 不是同一个问题

Validation/Test 变差并不总能简单归因于 Overfitting。

Overfitting 通常强调：模型过度适应有限训练数据中的细节，导致对同一目标分布的新样本表现较差。

Distribution Shift（分布偏移） 则强调：训练时和测试时的数据生成机制发生变化。例如：

- 训练是白天图片，测试是夜间图片；
- 训练 Agent 在环境版本 A，测试在环境版本 B；
- 训练文本来自普通网页，测试任务要求专业法律推理；
- 训练用户群体与实际部署群体不同。

如果 Train 和 Validation 根本不是同分布，那么 Validation Loss 高，可能既包含泛化不足，也包含 distribution shift。科研报告中应把这两种问题尽量区分。

一个常见错误

“Train 很好、Test 很差，所以一定是 Overfitting。”

不一定。也可能是 Train/Test 分布不一致、预处理不同、标签定义不同、数据泄漏修复后测试变难，甚至评估代码本身有 Bug。

7 Train / Validation 曲线：过拟合通常怎样出现？

最经典的过拟合过程是：

1. 训练初期，模型先学到比较稳定的主规律；

2. Train Loss 下降;
3. Validation Loss 也下降;
4. 训练继续进行, 模型开始越来越适应训练集的细节和噪声;
5. Train Loss 继续下降;
6. Validation Loss 反而开始上升。

本讲 Large MLP 的最佳验证点出现在:

$$Epoch = 111$$

此时:

$$Train\ MSE \approx 0.034981$$

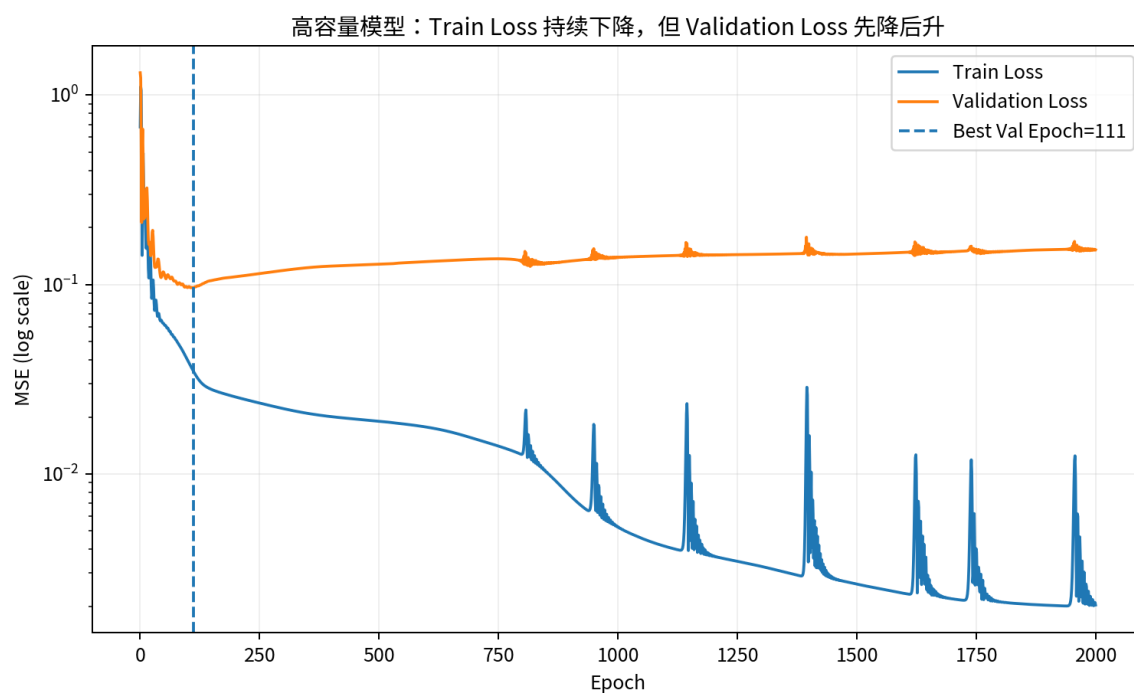
$$Validation\ MSE \approx 0.095893$$

继续训练到 2000 Epoch 后:

$$Train\ MSE \approx 0.002010$$

但:

$$Validation\ MSE \approx 0.152615$$



非常重要的判断

如果 Train Loss 继续下降, 而 Validation Loss 已经系统性恶化, 不能说“再训练一会儿应该会更好”。这通常是典型的过拟合信号之一。

8 Generalization Gap 是什么？

在同一个 Loss、同一套数据预处理口径下，可以定义一个简单的泛化差距：

$$Gap = Validation Loss - Training Loss$$

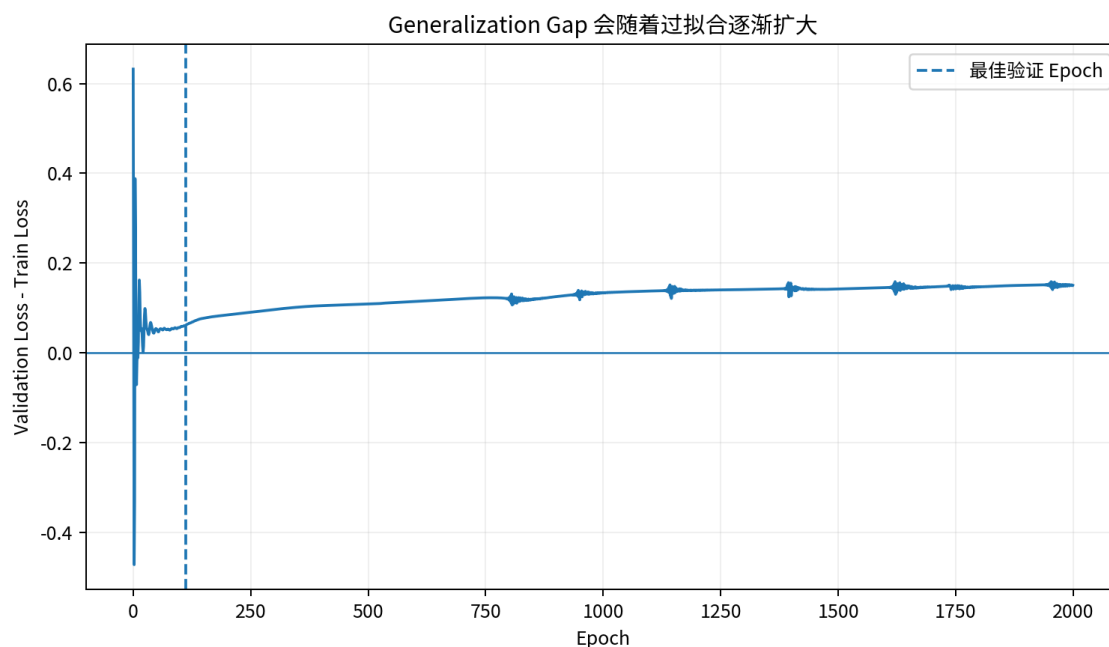
本讲 Large MLP 在最佳验证 Epoch：

$$Gap \approx 0.095893 - 0.034981 = 0.060912$$

训练到最后：

$$Gap \approx 0.152615 - 0.002010 = 0.150605$$

差距明显扩大。



不要机械解释 Gap

Gap 大常常提示泛化问题，但它不是万能指标。Train/Validation 数据难度不同、数据增强策略不同、Loss 口径不同、Dropout/BatchNorm 模式不同，都可能让数值不能直接简单相减。因此论文里看到 Generalization Gap，先检查比较口径是否一致。

9 为什么会 Overfit？不要只记“模型太大”

过拟合通常是多个因素共同作用。

因素 1：数据太少

如果只有很少训练样本，模型很难判断某个细节究竟是稳定规律还是偶然巧合。

因素 2：模型容量相对太高

Parameter 很多、模型表达能力强，但可约束它的数据证据很少，更容易学习训练集特有模式。

因素 3：标签或输入有噪声

噪声本来就不是可稳定预测的真实规律。高容量模型如果连噪声都拟合进去，Training Loss 会继续下降，而新样本不会共享同一份噪声。

因素 4：训练过久

在一些任务上，训练早期先学稳定模式，后期才逐渐记忆细节。于是 Epoch 本身也会成为一种“有效容量控制”。

因素 5：数据泄漏或 shortcut

这类情况尤其危险，因为 Validation/Test 也可能被污染，造成“看起来泛化很好”的假象。例如同一个人的多个高度相似样本被随机拆到 Train/Test，模型可能只是识别个体，而不是学习真正任务规律。

Agent Systems 中的对应风险

如果同一条长 trajectory 被切成多个片段，再随机分到 Train 与 Test，两个集合可能共享极强的上下文、环境状态或任务模板。这种情况下测试成绩高，不一定代表 Agent 能泛化到真正新任务或新环境。

10 Random Labels：最强的反例之一

如果标签本身完全随机、和输入没有真实关系，理论上不存在可以推广到新数据的规律。

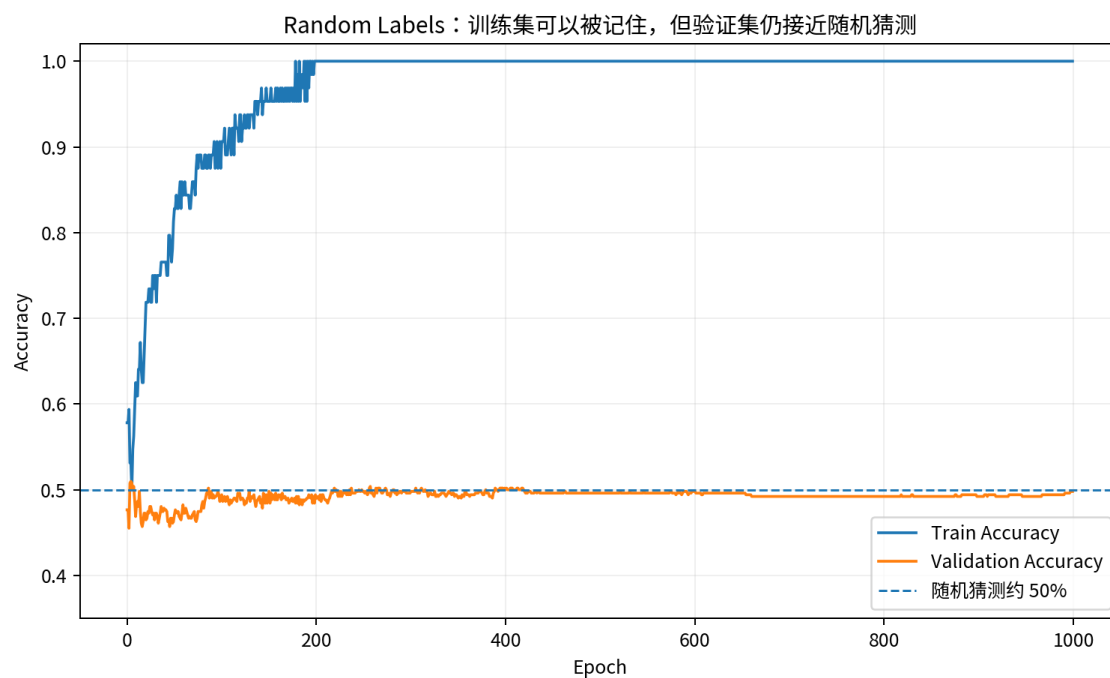
本讲实验随机生成二维输入，并随机分配二分类标签。高容量 MLP 训练 1000 Epoch 后：

$$\text{Train Accuracy} = 100\%$$

但是独立随机验证集：

$$\text{Validation Accuracy} \approx 49.8\%$$

二分类随机猜测本来就约为 50%。



这说明什么？

一个模型完全可以把训练样本记住到接近 100%，同时没有学到任何可以泛化的真实规律。因此：

Training Accuracy / Training Loss 本身不是 Generalization 的证据

11 Early Stopping: 为什么“最佳模型”经常不是最后一个 Epoch?

如果 Validation Loss 呈现先降后升，可以在验证集表现最好时保存模型：

$$\theta^* = \arg \min_{\theta_t} L_{val}(\theta_t)$$

这就是 Early Stopping 的基本思想。

本讲 Large MLP:

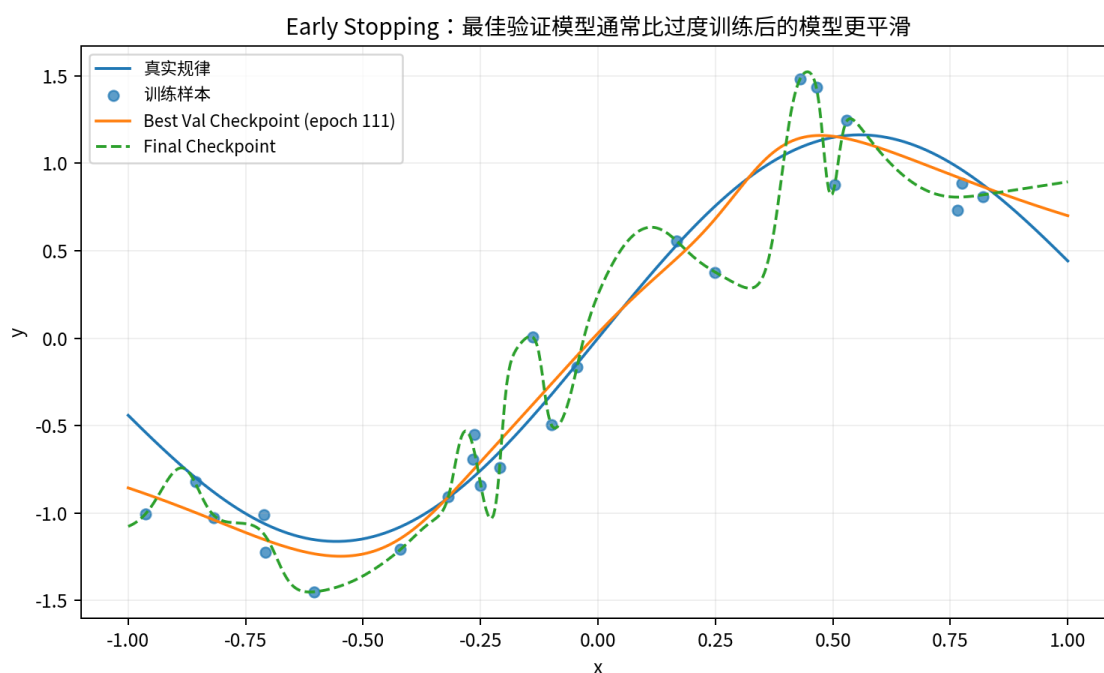
$$Best\ Validation\ Epoch = 111$$

最佳 checkpoint 在 Test 上:

$$Test\ MSE \approx 0.101043$$

训练到最后的 checkpoint:

$$Test\ MSE \approx 0.161832$$



实践中不会简单“只要上升一次就停止”

Validation Loss 本身会抖动，所以常见 Early Stopping 会加入：

- `patience`：允许连续若干 Epoch 没有改善；
- `min_delta`：改善幅度至少超过某个阈值才算真正改善；
- 保存 `best_state_dict`，而不是只保留最后模型。

一个简化逻辑：

```
best_val = inf
bad_epochs = 0
for epoch in range(max_epochs):
    train_one_epoch()
    val_loss = evaluate_validation()
    if val_loss < best_val - min_delta:
        best_val = val_loss
        save_checkpoint()
        bad_epochs = 0
    else:
        bad_epochs += 1
    if bad_epochs >= patience:
        break
```

Early Stopping 不是用 Test Set 调

应该由 Validation 决定什么时候停止、选哪个 checkpoint。Test Set 如果反复被拿来看、拿来改超参数，它实际上就逐渐变成了 Validation Set，最终测试结果会乐观偏置。

12 Validation 也会被“过拟合”：反复调参本身就是信息泄漏

初学时容易认为：只要 Test 不动，Validation 怎么用都可以。实际上，如果我们针对同一个 Validation Set 反复尝试几百个模型、超参数、prompt 或 stopping rule，然后只选其中最好的一次，模型设计过程本身就在不断吸收 Validation 的信息。

可以把每次调参理解成：

设计一个实验 → 看 Validation → 根据结果修改设计

重复很多次后，Validation 已经不再完全“陌生”。这叫做对 Validation 的 **selection overfitting** 或模型选择偏置。

因此成熟实验往往需要：

- Train：用于 Parameter 学习；
- Validation：用于有限度的模型选择与超参数选择；
- Test：尽量最后才使用；
- 重要结论最好再用外部数据集、时间外数据、不同 benchmark 或新环境复核。

这也是为什么一个 benchmark 被整个社区反复优化多年后，即使单篇论文没有直接训练 benchmark 样本，也要警惕社区层面的 benchmark overfitting。

13 Bias-Variance：只需要建立直觉，不做完整统计学习理论

当前阶段只需要一个简单直觉：

- 模型太简单：对真实规律表达不足，偏差高，容易 Underfit；
- 模型太灵活：对训练数据细节非常敏感，容易出现高方差和 Overfit；
- 数据更多、合理 Regularization、合适的模型容量，可以改善这种平衡。

不要把它机械理解成“某个模型一定是高 Bias 或高 Variance”。真正判断仍要回到具体数据分布、训练流程和独立评估。

14 Overfitting 一定表现为 Validation Loss 明显上升吗？

不一定。典型“先降后升”曲线非常有教学价值，但真实科研中可能出现更多形态：

1. Validation 仍在缓慢下降，但 Train 下降得更快，Gap 持续扩大；
2. Validation 指标平台化，继续训练几乎只改善 Train；
3. Validation 看起来正常，但某些子群体、长序列、难样本已经明显恶化；
4. IID Validation 很好，但时间外、环境外、任务外数据表现差；

5. 总体平均指标不变，但 Calibration、Robustness 或 Failure Mode 变差。

因此 Overfitting 不是一个只靠“看一条曲线”就能完全判断的二元标签。学习曲线是重要证据，但还要结合数据划分、子集分析、外部评估和失败案例。

15 Early Stopping 的工程细节：为什么要保存 best checkpoint

一个非常常见的错误是：程序虽然记录了 Validation 最佳 Epoch，但最后保存的却仍然是训练结束时的参数。

正确逻辑需要把当前最好的参数真正复制出来：

```
if val_loss < best_val:
    best_val = val_loss
    best_epoch = epoch
    best_state = copy.deepcopy(model.state_dict())
```

之所以需要 `deepcopy`，是因为后续 Optimizer 仍会继续原地修改 Parameter；如果只是保存一个引用，最后可能拿到的还是被后续训练覆盖后的状态。

如果还要严格恢复训练，而不是只做推理，通常还要一起保存：

- model state;
- optimizer state;
- scheduler state;
- 当前 epoch / global step;
- random seed / RNG state（严格复现时）;
- 数据采样器状态（某些场景）。

这和前面 Adam / AdamW 课程中讲过的 Optimizer State 是直接连接的。

16 怎样读 Train / Validation 曲线？

下面给出一个最低诊断框架。

现象	初步解释	下一步检查
Train 高，Val 也高	可能欠拟合或优化失败	容量、LR、训练时长、特征、Initialization
Train 低，Val 也低	当前验证分布上较好	Test、分布外数据、多个 seed、泄漏
Train 持续降，Val 先降后升	典型过拟合信号	Early stopping、regularization、数据量
Train 很低，Val 从一开始就很差	强过拟合或分布不一致	split、数据分布、leakage、shortcut
Train/Val 都剧烈震荡	不一定是泛化问题	LR、batch、数值稳定、评估样本量

17 一个完整的泛化诊断流程

以后做自己的实验，可以按下面顺序排查，而不是看到 Test 差就立即换模型。

Step 1: 先确认 Optimization 是否正常

检查：Training Loss 是否下降？Gradient 是否异常？LR 是否合理？如果 Training 都学不会，问题首先可能是优化失败，而不是 Overfitting。

Step 2: 检查 Train / Validation 的差距

如果 Train 明显优于 Validation，记录 Gap 是从什么时候开始扩大的，并观察是否存在最佳 Validation checkpoint。

Step 3: 检查数据划分

确认是否存在重复样本、同源样本、同一用户、同一 trajectory、同一任务模板跨 split 泄漏。

Step 4: 检查分布

比较 Train/Validation/Test 的类别比例、长度、时间段、环境版本、难度和预处理是否一致。

Step 5: 再尝试解决方案

可能措施包括：更多数据、数据增强、Early Stopping、Weight Decay、Dropout、减小容量、改进 split、修复 leakage、重新定义评价协议。

Step 6: 最后才下机制结论

如果某方法减少了 Gap，只能先说“在当前实验中改善了泛化指标”。要证明具体原因，需要额外 Ablation 和机制证据。

18 与 LLM / Transformer 的关系

在 LLM 中，Training Loss 常对应 next-token Cross Entropy：

$$L_{train} = -\frac{1}{M} \sum_{t=1}^M \log p_{\theta}(x_t | x_{<t})$$

Training Loss 降低说明模型越来越会预测训练语料里的 Token，但不能仅凭这一点判断它在新文本、新任务、Agent 环境里一定表现更好。

需要继续看：

- held-out validation perplexity / loss;

- 下游 benchmark;
- 分布外任务;
- 是否存在 benchmark contamination;
- 是否只是 memorization;
- 训练数据与评测数据是否高度相似。

为什么这对 Agent Systems 更重要?

Agent 的“样本”经常不是独立的一行表格，而是 trajectory、episode、tool trace、environment state。若拆分单位设计错误，就可能让未来信息、同任务模板或同一环境实例泄漏到 Train/Test 两侧。此时高分更像“识别见过的轨迹模式”，而不是泛化到新任务。

19 科研实验中怎样避免“把训练好看当成论文证据”?

至少做到：

1. **明确 split unit**：按样本、用户、任务、trajectory、时间还是环境拆分？
2. **Validation 与 Test 分工明确**：Validation 用于模型选择，Test 用于最终独立报告；
3. **报告 Train/Val/Test**：必要时同时画学习曲线；
4. **控制训练预算**：比较方法时尽量控制 steps、tokens、data、seed 等关键预算；
5. **多个 random seeds**：单次最好结果可能只是随机性；
6. **检查 leakage / contamination**：尤其是 LLM benchmark 和 Agent trajectory；
7. **不要从 Generalization Gap 单独推因果机制**：Gap 只能描述现象，不能直接证明为什么会发生。

20 把本讲实验代码映射回 PyTorch

本讲训练过程本质仍然是你已经熟悉的五步：

```
optimizer.zero_grad()
pred = model(x_train)
train_loss = loss_fn(pred, y_train)
train_loss.backward()
optimizer.step()
```

新增的是：每个 Epoch 之后不要只看 Training Loss，还要冻结参数更新，在独立 Validation 上评价当前 checkpoint。

```
model.eval()
with torch.no_grad():
    val_pred = model(x_val)
    val_loss = loss_fn(val_pred, y_val)
```

注意：Validation 只 Forward，不 `backward()`、不 `step()`，因为它不是用来训练 Parameter 的。

21 本讲总结：把“学会训练”升级成“学会判断模型是否真的学会”

最低闭环

看到一份深度学习实验时，至少要主动问：

1. Training Loss 是否下降？这是 Optimization 问题；
2. Validation/Test 是否同步改善？这是 Generalization 问题；
3. Train/Val gap 是否扩大？
4. 最佳 checkpoint 在哪里？是否只报告最后一个 Epoch？
5. Test 有没有被用于调参？
6. 数据划分有没有泄漏？
7. 高性能到底来自可泛化规律，还是 memorization / shortcut / contamination？

与后续课程的连接

第 42 讲会把 Train / Validation / Test 的职责和 Data Leakage 更严格地拆开；第 43–44 讲会进入 Regularization 与 Dropout，回答“已经发现 Overfitting 后，可以怎样约束模型”；第 45–47 讲进入 Normalization，解决的是另一类训练稳定性和尺度问题。不要把 Regularization、Normalization、Optimization 混成同一个概念。